



上海大学

SHANGHAI UNIVERSITY

本科毕业论文（设计）

UNDERGRADUATE THESIS (PROJECT)

题 目： 基于嵌入式平台的目标检测系统研究

学 院： 通信与信息工程学院

专 业： 电子信息工程

学 号： 22123739

学生姓名： 朱奕澄

指导教师： 滕国伟

起讫日期： 2025.12.29-2026.05.22



姓 名：朱奕澄

学号：22123739

论文题目：基于嵌入式平台的目标检测系统研究

原创性声明

本人声明：所呈交的论文是本人在指导教师指导下进行的研究工作。除了文中特别加以标注和致谢的地方外，论文中不包含其他人已发表或撰写过的研究成果。参与同一工作的其他同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

签 名：_____日 期：_____

本论文使用授权说明

本人完全了解上海大学有关保留、使用学位论文的规定，即：学校有权保留论文及送交论文复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容。

（保密的论文在解密后应遵守此规定）

签 名：_____指导教师签名：_____滕国伟_____日期：_____

摘要

本论文围绕一个具体部署问题展开：把已训练好的单类无人机检测模型迁移到 RK3588 开发板上，并在固定视频和 RTSP 实时链路中输出可观察的带框画面。围绕这一目标，本文完成了 PyTorch 权重、ONNX 模型到 RKNN 模型的转换，编写 C++17 板端检测程序，输出检测视频、CSV 日志和 RTSP 视频流。开发过程中重点处理了模型输出格式不匹配、后处理解析、实时推流延迟和检测框抖动等问题。

实验部分使用固定公开视频和实时 RTSP 两类输入对系统进行验证。结果显示，恢复后的单类无人机 RKNN 模型能够在 RK3588 上稳定产生有效检测框；双 context 适合展示逐帧检测下的 NPU 吞吐能力，而检测间隔、动态 ROI、运动预测和框平滑更适合系统演示场景下的低延迟实时显示。量化和硬件预处理实验也给出了边界条件：full INT8 在公开视频中出现置信度塌缩，hybrid INT8 恢复了检测但未稳定超过 FP 稳定方案；RGA 和 zero-copy 路径已完成端到端验证，但收益受到输入内存和推流链路影响。1 小时独立推理共完成 24,700 次 NPU 调用，未出现崩溃或明显内存泄漏。上述结果表明，RK3588 可以支撑无人机检测模型的端侧部署，但最终配置需要在检出稳定性、推理频率、输入拷贝和实时观看体验之间折中。

关键词：嵌入式平台；目标检测；RK3588；神经网络处理单元；多线程

ABSTRACT

This thesis starts from a practical deployment task: moving a trained single-class UAV detection model to an RK3588 development board and making the detection result visible in both fixed-video tests and real-time RTSP streams. Based on a YOLOv10 UAV model, the work completes the conversion from PyTorch and ONNX to RKNN, implements C++17 detection programs on the board, and exports annotated video, CSV logs, and RTSP video streams. During implementation, the main difficulties were not only model conversion, but also output tensor parsing, post-processing consistency, streaming latency, and bounding-box jitter.

The experiments use both public UAV videos and real-time RTSP input to check whether the system can run as a usable embedded application. The recovered RKNN model can produce detection boxes on RK3588, and multi-context inference increases throughput when every frame is detected. For real-time viewing, however, detection interval control, dynamic ROI, motion prediction, and box smoothing give a better balance between refresh rate and visual stability. The hardware optimization tests also exposed important limits. Full INT8 quantization caused confidence collapse in the public video test, while hybrid INT8 recovered detections but did not consistently outperform the FP path. RGA and zero-copy input were verified as switchable paths, but their end-to-end gain was still affected by memory transfer and streaming overhead. In a one-hour standalone inference test, the system completed 24,700 NPU runs without crashes or obvious memory leakage. These results show that RK3588 can support edge deployment of the UAV detector, but the final demonstration configuration must be chosen by considering detection stability, input transfer, inference interval, and viewing latency together.

Keywords: Embedded platform; Object detection; RK3588; Neural Processing Unit; Multithreading

目 录

第一章 绪论	6
1.1 研究背景与意义	6
1.2 国内外研究现状	6
1.3 本文主要研究内容	7
1.4 论文组织结构	8
第二章 相关技术基础	9
2.1 目标检测算法基础	9
2.2 无人机小目标检测特点	9
2.3 RK3588 嵌入式平台	10
2.4 C++17 多线程与实时视频流水线	11
2.5 本章小结	11
第三章 系统总体设计	12
3.1 系统需求分析	12
3.1.1 功能需求	12
3.1.2 性能需求	12
3.1.3 部署约束	13
3.2 系统总体架构	13
3.3 模型移植与检测模块设计	14
3.3.1 模型移植流程	14
3.3.2 检测结果解析	15
3.4 实时视频处理流水线设计	16
3.4.1 多线程流水线结构	16
3.4.2 实时输出方式	16
3.5 动态 ROI 与轻量跟踪策略设计	17
3.5.1 检测间隔策略	17
3.5.2 动态 ROI 策略	17
3.5.3 轻量跟踪与框平滑	17
3.6 多 context NPU 并行推理设计	18
3.6.1 多 context 设计思想	18

3.6.2 多 context 与检测间隔的关系	19
3.6.3 阶段耗时分析设计	19
3.7 本章小结	20
第四章 系统实现与调试	21
4.1 开发与部署环境	21
4.2 模型导出与 RKNN 转换实现	22
4.3 检测程序实现	25
4.4 检测间隔、动态 ROI 与轻量跟踪实现	27
4.5 多 context NPU 并行推理实现	29
4.6 日志记录与性能调试接口	31
4.7 本章小结	32
第五章 系统测试与实验分析	33
5.1 实验环境与评价指标	33
5.2 模型训练与测试	34
5.3 检测间隔与框平滑策略实验	38
5.4 性能优化与瓶颈分析	39
5.5 长时间运行稳定性测试	44
5.6 本章小结	44
总结与展望	46
参考文献	47
致谢	49

第一章 绪论

1.1 研究背景与意义

本论文的出发点不是单纯验证 YOLO 模型能否检测无人机，而是将已经训练好的无人机检测模型迁移到 RK3588 开发板上，并让它在本地视频和 RTSP 实时流中稳定输出带框画面。实际调试过程中，模型能否被 RKNN Runtime 正确加载、输出张量是否与后处理代码匹配、视频推流是否产生累计延迟，都会直接影响最终演示效果。

从应用背景看，无人机在低空巡检、公共安全、应急救援和城市管理等场景中使用越来越多，也带来了目标监测、区域安全和实时预警等需求。如果将视频流全部上传到云端处理，系统会受到网络带宽、传输延迟和隐私风险的限制。相比之下，将检测模型部署到靠近摄像头的嵌入式设备上，可以在本地完成视频采集、图像预处理、模型推理和结果输出，更适合低延迟演示和现场应用。

因此，本文围绕 RK3588 平台上的无人机检测系统展开研究。本文重点不在于重新提出新的目标检测网络结构，而是在现有单类无人机检测模型基础上完成模型迁移、C++17 部署、多线程视频流水线、NPU 多 context 推理和性能分析。这样的研究更关注算法从训练环境走向边缘设备时遇到的工程问题，也为后续同类目标检测系统的板端部署提供可复现的实现经验。

1.2 国内外研究现状

目标检测是计算机视觉领域的重要任务之一，其目标是在图像或视频中定位并识别特定类别目标。以 YOLO 系列为代表的一阶段检测算法具有结构简洁、推理速度快和工程部署方便等特点，在实时检测任务中应用广泛^[1-3]。随着网络结构、特征融合和训练策略不断改进，目标检测算法在通用数据集上的精度和速度均有明显提升。

无人机检测属于小目标检测的典型应用场景。远距离无人机在图像中往往只占据少量像素，且容易受到天空、建筑、树枝、鸟类和光照变化等干扰。针对这类任务，研究者通常从高分辨率特征、注意力机制、跨尺度融合、样本增强和专用数据集构建等方面提升检测能力^[4-5]。相关研究表明，面向反无人机场景的检测算法正逐步关注小目标表征、轻量化结构和边缘部署友好性。

近年来，已经有了很多针对无人机小目标检测和边缘部署的改进工作。何宇豪等通过额外检测头、BiFPN、CBAM 和 Soft-NMS 提高无人机航拍小目标检测效果^[22]；韩俊等围绕密集小目标提出多尺度特征、注意力和自适应融合结构^[23]；何植仟和曹立杰面向无人机航拍图像提出 UAVAI-YOLO，引入 P2 特征层和改进特征融合以提升小目标检测能力^[24]；廖嘉仪进一步面向嵌入式边缘计算设备讨论反无人机检测中的精度与速度平衡问题^[25]。这些工作说明，无人机检测研究正在从单纯提高精度转向小目标表征、轻量化结构和端侧部署的协同优化。

在嵌入式部署方面，研究重点已经从单纯模型压缩逐步转向系统级协同优化^[6-7]。本文的实验现象也说明，板端实时帧率并不只由 `rknn_run` 耗时决定，图像预处理、数据拷贝、后处理、编码推流、线程同步和队列积压都会改变最终观看效果。因此，本文在评价系统时同时记录 `stream_fps`、`npu_fps`、端到端延迟、CPU 占用和 RSS 内存，而不是只报告单帧 NPU 推理耗时。

1.3 本文主要研究内容

本文围绕 RK3588 嵌入式平台上的无人机目标检测系统开展研究，主要内容包括以下几个方面：

（1）完成无人机检测模型从训练环境到 RK3588 平台的迁移。本文以已训练的 YOLOv10 单类无人机检测模型为基础，完成 PyTorch 权重、ONNX 模型和 RKNN 模型之间的转换与验证，并对板端输出张量进行解析。基础检测模型由同组负责算法设计与训练的组员协同提供，本文重点负责该模型面向 RK3588 平台的模型格式转换、算子适配、板端后处理和多线程系统级部署验证。

（2）设计并实现 C++17 固定视频检测和实时 RTSP 检测程序。固定视频程序用于可重复实验和公开视频验证，实时 RTSP 程序用于 USB 摄像头或本地视频输入下的实时显示。

（3）构建面向实时检测的多线程处理流程。系统将视频读取、NPU 推理、结果绘制、RTSP 推流和日志记录进行解耦，降低单一串行流程对实时性的影响。

（4）实现检测间隔、动态 ROI、轻量跟踪和框平滑策略。上述策略用于降低不必要的 NPU 调用，并改善实时观看时检测框抖动和延迟问题。

（5）开展 RK3588 NPU 多 context 并行推理实验。本文比较单 context、双 context

和多 context 配置下的运行表现，并结合阶段耗时分析定位系统瓶颈。

1.4 论文组织结构

本文共分为六章，具体安排：

第一章介绍研究背景、研究意义、国内外研究现状和本文主要研究内容。

第二章介绍目标检测、无人机小目标检测、RK3588 平台、RKNN 部署和 C++17 多线程视频处理等相关技术基础。

第三章给出系统总体设计，包括需求分析、总体架构、模型迁移设计、实时视频处理流水线、动态 ROI 与多 context 推理设计。

第四章阐述系统实现与调试过程，包括开发环境、模型转换、RKNN 推理模块、固定视频程序、实时 RTSP 程序和日志接口。

第五章给出系统测试与实验分析，包括训练结果、公开视频固定输入验证、实时 RTSP 实验、多 context 并行实验和阶段耗时分析。

第六章总结全文工作，并对 INT8 量化、完整 RGA 硬件预处理闭环、GPIO 闭环控制和真实飞行测试等后续方向进行展望。

第二章 相关技术基础

2.1 目标检测算法基础

目标检测任务需要同时完成目标类别判断和边界框定位。与图像分类任务相比，目标检测不仅需要判断图像中是否存在某类目标，还需要给出目标在图像中的空间位置。现代目标检测算法通常由主干网络、特征融合模块和检测头组成。主干网络负责提取图像特征，特征融合模块用于整合不同尺度信息，检测头输出目标框位置、类别和置信度。

本课题选择 YOLOv10 并不是为了比较所有检测网络的优劣，而是因为前期已经获得了单类 drone 模型，且其输出形式便于在板端重新实现后处理。对本系统而言，检测算法最终要落在两个接口上：输入张量必须能被 RKNN Runtime 接收，输出张量必须能还原成坐标框和置信度。后续调试中，end2end 导出方式与 RKNN 端侧后处理并不完全匹配，因此本文采用 end2end=False 的导出方式，在 C++17 程序中完成置信度筛选、坐标还原和 NMS 处理。

以 640×640 输入为例，YOLO 类模型通常在 80×80 、 40×40 和 20×20 等尺度上产生候选位置。这个设计对小目标检测很关键，因为远距离无人机在图像中往往只占少量像素。本文实际使用的单类模型输出为 $1 \times 5 \times 8400$ ，其中 8400 正对应三种尺度候选位置之和。这样写后处理代码时，可以直接围绕中心点、宽高和置信度五个通道进行解析，而不必保留完整 COCO 类别表。

部署到 RK3588 后，模型精度只是问题的一部分。实际影响画面效果的还有输入拷贝、颜色空间转换、后处理耗时和 RTSP 发布延迟。也正因为如此，本文没有只记录单帧推理时间，而是把 stream_fps、npu_fps、端到端延迟和 RSS 内存占用一起作为实验指标。

2.2 无人机小目标检测特点

本项目调试中，小目标问题首先表现为阈值敏感和检测框抖动。公开视频中的无人机有时只有很小一块区域，置信度稍有波动就会影响是否产生有效检测框；检测框位置偏移几像素，也会让 IoU 评价明显下降。鸟类、云层、树枝和建筑边缘还会带来

相似干扰，因此无人机检测不能只看模型在单张清晰图片上的表现。

实时性要求在系统演示中更加直观。固定视频实验可以慢一些，但必须可重复、可记录；实时 RTSP 演示则要求画面连续、框不要明显左右跳、延迟不要累积。基于这个考虑，后续章节把检测间隔、框平滑、动态 ROI 和多 context 推理都作为可切换策略进行测试。

2.3 RK3588 嵌入式平台

RK3588 被选为本文部署平台，主要原因是它同时具备 CPU、NPU、视频编解码和较完整的 Linux 开发环境。根据 Rockchip 公开规格资料，该平台采用 4 个 Cortex-A76 性能核心与 4 个 Cortex-A55 能效核心组成的 big.LITTLE 架构，集成 Mali-G610 MC4 GPU，并提供标称 6 TOPS (INT8) 的 NPU 算力^[12]。这些硬件资源为视频采集、图像预处理、模型推理和结果输出提供了基础。

不过，在本文的测试中，NPU 理论算力并不能直接等同于最终观看帧率。CPU 仍要负责视频读取、张量组织、后处理、线程调度和日志记录；如果输入设置或推流环节出现等待，NPU 本身再快也不能完全消除端到端延迟。因此，本文后续实验会同时分析 rknn_run、rknn_inputs_set 和输出发布等阶段。

内存和视频输入输出能力同样影响实验结果。RK3588 支持 LPDDR4、LPDDR4X 和 LPDDR5 等内存类型，较高带宽有助于降低图像拷贝和输入张量更新的等待时间^[12]。平台也提供 H.264、H.265 等硬件编解码能力，但本文当前默认运行方案仍以 OpenCV/GStreamer 链路为主，因此视频解码、颜色转换、缩放和推流都会参与最终耗时。

从同类边缘平台看，Jetson Nano 主要依赖 128 核 Maxwell GPU 完成深度学习推理，官方资料给出的 AI 性能为 472 GFLOPS^[13]；Jetson Orin Nano Super 的理论 AI 算力可达到 67 INT8 TOPS^[14]。相比之下，RK3588 的绝对算力并不是最高，但它的成本、功耗、接口资源和国产嵌入式设备适配性更符合本课题的实验条件。

实际部署时，一个视频帧从输入到显示会经过多个环节：解码、缩放、颜色转换、输入张量设置、NPU 推理、输出解析、绘制和推流。任何一个环节耗时过高，都可能造成队列积压。后续的阶段耗时实验正是为了判断瓶颈到底出在 NPU 计算、输入拷贝还是视频输出，而不是凭感觉选择优化方向。

在本论文中，RKNN 部署不是一次格式转换就结束。训练得到的 PyTorch 权重需要先导出为 ONNX，再通过 RKNN Toolkit2 转换为 RKNN 模型[7,9]。因此，系统需要建立从训练模型到 RKNN 模型的转换流程，并在板端验证模型输入输出张量是否与后处理代码匹配。本文当前稳定运行方案采用浮点 RKNN 模型，同时补充实现了 RGA 硬件预处理闭环和 hybrid INT8 量化实验路径，用于验证任务书中硬件预处理与量化优化相关要求。

2.4 C++17 多线程与实时视频流水线

实时视频检测系统通常包含多个耗时不同的处理阶段。如果把视频读取、图像预处理、NPU 推理、后处理和视频输出全部放在同一条串行流程中，最终帧率会被最慢的阶段限制，画面也容易出现延迟累积。多线程流水线的作用，是把这些阶段拆开，让不同线程围绕相邻视频帧并行工作。

本文选择 C++17 实现板端程序，主要是为了直接调用 RKNN Runtime、OpenCV、GStreamer 和 RGA 等底层接口，并对线程、队列和内存生命周期进行更明确的控制。具体实现中，采集线程负责持续写入最新帧，推理线程按照当前策略决定是否调用 NPU，输出线程负责绘制检测结果并发布视频流。该设计并不追求每一帧都必须完整检测，而是优先保证实时观看时画面连续、延迟可控。

2.5 本章小结

本章从模型、平台和实时视频链路三个角度梳理了本文系统实现所需的技术基础。对本文而言，YOLO 模型的关键不只在检测精度，还在于导出格式和板端后处理是否可控；RK3588 的关键也不只在 NPU 理论算力，还在于输入设置、视频处理和线程调度能否与推理过程配合。基于这些分析，后续章节将重点说明模型迁移、系统流水线和多 context 推理策略的设计与实现。

第三章 系统总体设计

基于第二章讨论的技术基础，本章将重点放在系统本身的设计上。本文的目标不是重新提出一种检测网络，而是在 RK3588 上实现一套可以输入视频、调用 NPU 推理、输出带框画面并记录实验结果的无人机检测系统。因此，本章先从功能、性能和部署约束出发，说明系统为什么要采用固定视频验证和实时 RTSP 演示两条路径。

与 PC 端离线检测不同，嵌入式部署需要同时考虑模型格式、输入输出张量、推理延迟、线程同步和视频发布等问题。后续的系统架构、检测间隔、动态 ROI、轻量跟踪和多 context 推理设计，都是围绕“板端稳定运行”和“实时观看体验”这两个目标展开。

3.1 系统需求分析

3.1.1 功能需求

本文系统面向 RK3588 嵌入式平台上的无人机目标检测应用，首先需要支持本地视频、公开视频和 USB 摄像头等输入源，并能够在输出画面中绘制目标框、类别名称和置信度。为了支撑论文中的对比实验，系统还需要保存逐帧检测结果、运行帧率、推理耗时、ROI 记录和报警事件等日志，使同一段视频能够在不同配置下重复测试。

除了固定视频测试外，系统还需要提供实时 RTSP 输出。这样在板端连接摄像头后，电脑端可以通过 RTSP 地址查看带框检测画面，便于演示和调试。固定视频路径强调可重复和可对比，实时 RTSP 路径强调低延迟显示，两者共同构成本文的验证入口。

3.1.2 性能需求

嵌入式实时检测不能只看模型单次推理时间。实际画面效果还会受到视频读取、颜色转换、输入张量设置、后处理、绘制、编码推流和队列调度影响。因此，本文将 `stream_fps`、`npu_fps`、端到端延迟、CPU 占用、RSS 内存和丢帧情况一起作为实验指标，用来判断系统是否真正适合实时演示。

对于无人机小目标场景，检测框稳定性同样重要。公开视频中的目标往往尺寸较小，置信度波动会导致检测框时有时无；如果相邻帧框的位置变化过大，即使平均帧

率不低，观看效果也会显得抖动。因此，系统需要支持检测间隔、框平滑、动态 ROI 和轻量跟踪等策略，在检测刷新频率和显示稳定性之间取得平衡。

3.1.3 部署约束

本文系统的部署对象为 RK3588 嵌入式平台。该平台具备多核 CPU、GPU 和 NPU 等计算资源，但视频采集、图像预处理、NPU 推理、后处理和视频输出之间仍然存在明显的调度关系。如果各模块串行执行，任一环节耗时过长都会影响实时显示效果。因此，系统需要采用多线程流水线结构，将视频采集、模型推理和结果输出进行解耦。

模型格式也是部署约束之一。训练阶段通常得到 PyTorch 权重文件或 ONNX 模型，而 RK3588 的 NPU 推理需要使用 RKNN 格式模型。因此，系统需要建立从训练模型到 RKNN 模型的转换流程，并在板端验证模型输入输出张量是否与后处理代码匹配。本文当前稳定运行方案采用浮点 RKNN 模型，同时补充实现 RGA 硬件预处理和 hybrid INT8 量化实验路径，用于验证任务书中硬件预处理与量化优化相关要求。

3.2 系统总体架构

本文系统采用“视频输入层、预处理层、NPU 推理层、后处理层、策略控制层和结果输出层”的分层架构，如图 3.1 所示。视频输入层负责从本地文件、摄像头或 RTSP 输入中获取图像帧；预处理层负责完成图像缩放、颜色空间转换和输入张量组织；NPU 推理层负责调用 RKNN Runtime 完成神经网络前向计算；后处理层负责解析模型输出并生成检测框；策略控制层负责检测间隔、结果复用、动态 ROI、框平滑和轻量跟踪；结果输出层负责视频显示、RTSP 推流和日志保存。

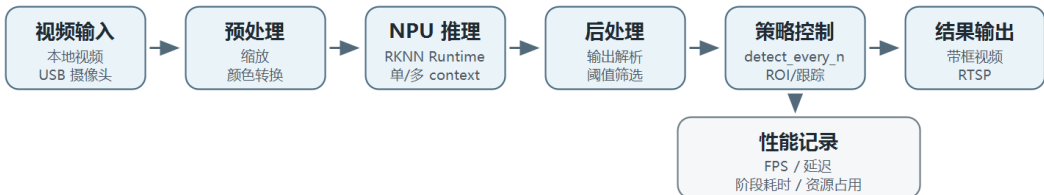


图 3.1 系统总体架构图

在系统运行过程中，视频帧首先进入采集线程。采集线程不直接等待 NPU 推理完成，而是将帧数据写入有限长度队列。推理线程从队列中取出图像帧，按照当前策略决定是否执行完整 NPU 检测。如果当前帧需要检测，则调用 RKNN 模型并更新检测结果；如果当前帧不需要检测，则根据上一帧检测框和轻量跟踪结果进行复用。输出线程负责将最新检测结果绘制到画面中，并通过视频文件或 RTSP 服务输出。

这种设计将视频输入、模型推理和显示输出解耦，避免单个慢速模块阻塞整个系统。对于固定视频实验，系统可以保存带框输出视频和逐帧检测结果；对于实时演示，系统可以通过 RTSP 地址在电脑端查看检测画面。固定视频实验保证可复现性，实时视频流验证实际部署效果，两者共同支撑后续实验分析。

3.3 模型移植与检测模块设计

3.3.1 模型移植流程

本文的目标检测模型首先在 PC 或 WSL 环境中完成训练和导出，然后转换为 RK3588 平台可加载的 RKNN 模型^[3,7-9]。整体流程如图 3.2 所示。



图 3.2 模型移植流程图

在模型移植过程中，关键问题不是简单完成格式转换，而是保证模型输出与板端后处理逻辑一致。本文当前使用的稳定模型为单类别 **drone** 检测模型，板端输出张量形式为 $1 \times 5 \times 8400$ ，其中 5 个通道对应边界框参数和置信度信息。8400 来自 YOLOv10 三个检测头的特征图位置数量之和，即 80×80 、 40×40 和 20×20 三个尺度共 8400 个候选位置。这种多尺度输出能够同时覆盖较大目标和远距离小目标，对无人机这类尺寸变化明显的目标具有实际意义。

为提高后处理模块的可复用性，本文在实现时没有将输出格式完全固定为单一模

型。后处理逻辑兼容 COCO 通用检测模型常见的 84 通道输出格式、单类别模型的 5 通道输出格式，以及端到端检测模型可能出现的 6 通道输出格式。其中，84 通道通常对应 4 个边界框参数和 80 个类别分数；5 通道对应单类目标框参数和置信度；6 通道端到端输出通常直接给出边界框、置信度和类别编号。通过在加载和解析阶段识别不同输出布局，系统能够在不重写整套视频处理流程的情况下切换模型，降低后续模型替换和对比实验的工程成本。

对于无人机检测任务，类别数量较少，后处理重点主要集中在置信度阈值、坐标还原和非极大值抑制上。由于目标在远距离情况下通常较小，阈值设置过高容易造成漏检，阈值设置过低则会增加误检。因此，系统将置信度阈值和 NMS 阈值设计为可配置参数，在不同视频场景中进行调试和记录。

3.3.2 检测结果解析

检测模块的输入为预处理后的图像张量，输出为若干候选框。每个候选框包含左上角坐标、右下角坐标、类别编号和置信度。由于当前模型为单类别无人机检测模型，因此类别编号统一映射为 `drone`。在结果可视化时，系统将类别名称和置信度绘制在检测框附近，便于观察模型输出。

检测结果解析过程包括以下步骤：

- （1）读取 RKNN 输出张量，并按照模型输出格式转换为候选框；
- （2）根据置信度阈值过滤低置信度候选框；
- （3）将网络输入尺寸下的坐标还原到原始图像尺寸；
- （4）对重叠候选框执行非极大值抑制；
- （5）输出最终检测框，用于绘制、日志保存和后续跟踪。

从设计角度看，检测后处理模块需要保持独立，避免与视频输入、RTSP 输出和多线程调度强耦合。这样做的好处是：当后续更换模型结构、输入尺寸或类别数量时，只需要调整模型加载和后处理逻辑，而不需要重写整个视频处理系统。

3.4 实时视频处理流水线设计

3.4.1 多线程流水线结构

为了提高实时性，本文系统采用多线程流水线设计。系统主要包含采集线程、推理线程和输出线程。采集线程负责从视频源读取帧；推理线程负责执行预处理、NPU推理和后处理；输出线程负责绘制结果并编码输出视频流。多线程结构如图 3.3 所示。

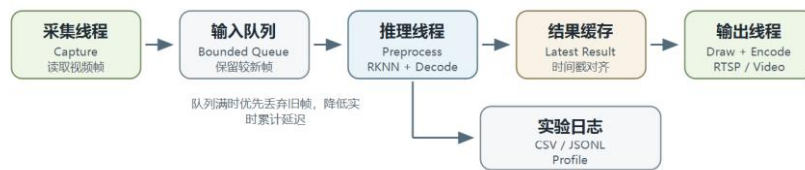


图 3.3 实时视频处理流水线

在该结构中，输入队列采用有限长度设计。当推理速度低于视频输入速度时，系统优先丢弃过旧帧，保留较新的输入帧。这样可以降低实时画面的累计延迟，避免系统为了处理历史帧而导致显示严重滞后。对于实时演示场景，低延迟通常比完整处理每一帧更重要；对于固定视频评测场景，则可以通过离线模式保存完整结果。

3.4.2 实时输出方式

本文系统支持两类输出方式。第一类是固定视频输出，即将检测框绘制到视频文件中，并保存逐帧检测结果。这种方式适合论文实验，因为同一输入视频可以在不同参数配置下重复运行。第二类是实时 RTSP 输出，即将带框画面编码为视频流，电脑端可以通过播放器打开 RTSP 地址进行观看。这种方式适合系统演示和板端联调。

固定视频和实时 RTSP 的定位不同。固定视频用于严谨对比，能够得到稳定的帧率、延迟和检测结果统计；实时 RTSP 用于验证系统是否能在实际部署中持续运行。本文后续实验中，固定视频作为主要对比入口，实时 RTSP 作为最终演示入口。

3.5 动态 ROI 与轻量跟踪策略设计

3.5.1 检测间隔策略

在嵌入式实时检测中，如果每一帧都执行完整 NPU 推理，系统可能出现延迟升高或视频卡顿。为此，本文设计了检测间隔策略，即每隔 N 帧执行一次完整检测，中间帧复用上一帧检测结果或通过轻量跟踪进行更新。该策略通过参数 `detect_every_n` 控制。

设当前帧编号为 k ，检测间隔为 N ，则是否执行完整检测可表示为：

$$\text{Infer}(k) = \{ 1, \quad k \bmod N = 0; \quad 0, \quad k \bmod N \neq 0 \} \quad (3.1)$$

当 $\text{Infer}(k)=1$ 时，系统执行完整 NPU 检测；当 $\text{Infer}(k)=0$ 时，系统不调用 NPU，而是根据上一帧结果和跟踪策略生成当前帧的显示框。该方法能够降低 NPU 调用频率，从而改善输出画面的连续性和系统延迟。

3.5.2 动态 ROI 策略

无人机目标在画面中通常较小，并且在相邻帧之间位置变化具有连续性。基于这一特点，系统设计了动态 ROI 策略：当上一帧存在稳定检测结果时，系统可以围绕目标位置生成局部区域，用于后续帧的跟踪、结果复用或局部分析。动态 ROI 的作用不是替代模型检测，而是在检测间隔期间提供目标区域约束，减少检测框跳变。

动态 ROI 的设计需要考虑两个问题。第一，ROI 区域不能过小，否则目标快速运动时容易丢失；第二，ROI 区域不能过大，否则对抑制背景干扰的作用会减弱。因此，系统在检测框周围保留一定扩展边界，并结合图像边界进行裁剪，保证 ROI 始终位于有效图像范围内。

3.5.3 轻量跟踪与框平滑

在实时显示中，即使模型能够正确识别目标，检测框也可能因为输入画面噪声、目标尺寸较小和模型置信度波动而产生抖动。为改善显示效果，本文系统设计了轻量级跟踪和平滑策略。跟踪策略用于在未执行完整检测的帧中估计目标位置；框平滑策

略用于减少相邻检测框之间的突变。

本文实现的轻量跟踪包含运动预测和光流跟踪两种模式。在 `TrackMode::kMotion` 模式下，系统根据连续两次检测结果的检测框中心点位移向量进行线性运动预测，并将预测位移外推到未执行完整检测的帧上。为避免小目标检测框偶发跳变导致显示框快速漂移，程序对单步位移量设置约束上限，并结合边界裁剪和周期性全帧检测进行修正。该方法计算量较小，不依赖额外训练模型，适合目标运动较平稳且检测间隔较短的实时演示场景。

在 `TrackMode::kOpticalFlow` 模式下，系统采用 Lucas-Kanade 稀疏光流方法估计目标区域运动^[15]。具体而言，系统首先在上一帧检测框或动态 ROI 区域内利用 `goodFeaturesToTrack` 提取 Shi-Tomasi 角点特征^[16]，然后调用 `cv::calcOpticalFlowPyrLK` 计算这些特征点在当前帧中的对应位置。为了抑制异常点影响，系统不直接使用所有光流位移的平均值，而是计算有效特征点位移的中值，并将该中值位移作用到上一帧检测框上。当有效特征点数量不足或光流跟踪失败时，系统退回到运动预测与上一帧检测框复用策略，从而保证显示过程连续。与 DeepSORT 等完整多目标跟踪框架相比^[19]，本文跟踪模块仅服务于检测间隔期间的显示框连续性，不承担跨目标身份保持任务。

框平滑采用指数平滑思想，将当前检测框与上一帧显示框进行加权融合：

$$B_smooth(k) = \alpha B_detect(k) + (1 - \alpha) B_smooth(k - 1) \quad (3.2)$$

其中， $B_detect(k)$ 表示当前帧检测框， $B_smooth(k-1)$ 表示上一帧平滑后的检测框， α 为平滑系数。 α 越大，检测框越接近当前检测结果； α 越小，检测框越稳定但响应速度越慢。本文在实际测试中将平滑参数设计为可配置项，用于在稳定性和响应速度之间取得平衡。

3.6 多 context NPU 并行推理设计

3.6.1 多 context 设计思想

RK3588 平台的 NPU 具备并行计算能力，但在单线程、单 context 调用方式下，视频预处理、输入设置、NPU 执行和输出读取往往串行发生，无法充分重叠不同帧之

间的处理过程。为了提高 NPU 使用效率，本文设计了多 context 并行推理结构，即创建多个 RKNN context，并由多个推理工作线程并行处理不同帧。多 context 并行推理结构如图 3.4 所示。

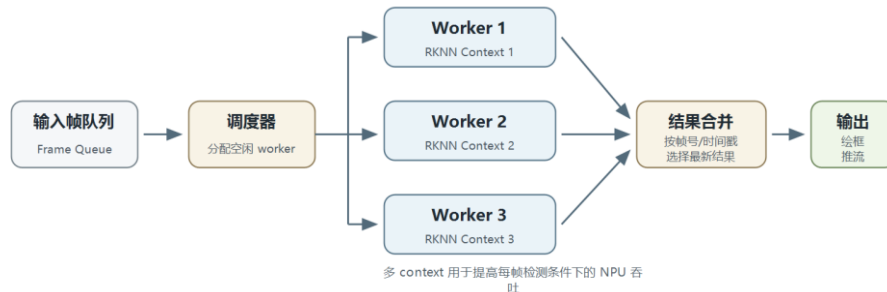


图 3.4 多 context NPU 并行推理结构

在该结构中，每个推理线程拥有独立的 RKNN context，线程之间不共享模型运行状态。输入帧由调度器分配给空闲 worker，推理完成后将结果写入结果队列。输出端根据时间戳或帧编号选择最新结果进行绘制，避免较旧推理结果覆盖较新画面。

3.6.2 多 context 与检测间隔的关系

检测间隔策略和多 context 策略解决的是不同问题。检测间隔通过减少 NPU 调用次数降低系统负载，适合实时演示中对低延迟和稳定显示的需求；多 context 策略通过并行化提高 NPU 推理吞吐，适合分析 RK3588 的硬件能力和每帧检测场景。

在实际设计中，两种策略可以组合使用。例如，当 detect_every_n=2 时，系统每两帧执行一次完整检测，可以获得较好的显示稳定性；当 detect_every_n=1 且启用两个 context 时，系统可以测试每帧检测条件下的最高 NPU 吞吐能力。本文后续实验将比较单 context、双 context 和多 context 的差异，分析不同配置对 stream_fps、npu_fps 和端到端延迟的影响。

3.6.3 阶段耗时分析设计

为了定位系统瓶颈，本文在推理模块中设计了阶段耗时记录功能。每次推理过程被划分为图像准备、输入设置、NPU 运行、输出读取、检测解码和资源释放等阶段。通过记录各阶段耗时，可以判断瓶颈来自 NPU 计算、输入数据传输、后处理还是视

频输出。

阶段耗时分析的意义在于避免盲目优化。例如，如果后处理耗时很低，而输入设置和 NPU 运行耗时占主要比例，则继续优化 NMS 或绘框代码对整体实时性提升有限；此时更值得关注输入格式、数据传输、多 context 调度或模型结构本身。本文后续实验将利用该设计对 RK3588 板端运行过程进行量化分析。

3.7 本章小结

本章确定了系统实现中的三个主要取舍：固定视频路径用于保证实验可重复，实时 RTSP 路径用于验证演示链路；检测间隔和框平滑用于降低实时观看延迟，多 context 用于测试每帧检测下的 NPU 吞吐；阶段耗时记录用于判断瓶颈是否出在 NPU、输入设置或视频输出。

这些设计不是孤立的模块堆叠，而是为后续板端实现和实验对比服务。第四章将在此基础上说明 RKNN 模型加载、视频流处理、检测后处理、RTSP 输出、多线程调度以及实验日志记录等内容如何在代码中落地。

第四章 系统实现与调试

在完成系统总体架构设计后，本章进入板端工程实现部分。实现工作的重点不是单个算法函数，而是把模型转换、RKNN 推理、视频输入输出和多线程调度连接成可以运行的程序。为便于后续实验，本文同时保留固定视频检测路径和实时 RTSP 检测路径：前者用于可重复验证，后者用于实时演示。

本章所述实现主要包括开发环境配置、模型导出与 RKNN 转换、RKNN 推理模块封装、固定视频检测程序、实时 RTSP 检测程序、检测间隔控制、动态 ROI、轻量跟踪、多 context 并行推理和阶段耗时记录等内容。上述模块共同构成后续性能实验的运行基础。

4.1 开发与部署环境

系统开发过程分为 PC/WSL 端和 RK3588 板端两部分。PC/WSL 端主要用于模型训练、模型导出、RKNN 转换准备和代码整理；RK3588 板端主要用于实际模型加载、视频输入、NPU 推理、检测结果绘制和 RTSP 输出。这样的分工可以降低板端开发压力，使耗时较长的模型训练和格式转换尽量在 PC 端完成，而把板端工作集中在实时推理和系统集成上。表 4.1 给出了本文系统实现所涉及的主要软硬件环境。

表 4.1 系统开发与部署环境

类别	环境或工具	作用说明
开发主机	Windows 与 WSL 环境	用于代码管理、模型训练脚本运行、ONNX/RKNN 转换准备和论文资料整理
嵌入式平台	RK3588 开发板	作为最终部署平台，利用板载 NPU 执行目标检测推理
目标检测模型	YOLOv10 单类无人机检测模型	作为系统目标检测算法主体，输出无人机候选框和置信度
推理框架	RKNN Runtime	在 RK3588 板端加载 RKNN 模型并调用 NPU 推理
视频处理库	OpenCV	用于视频文件读取、USB 摄像头读取、图像缩放、颜色转换和检测框绘制
推流组件	GStreamer RTSP Server	用于将检测后的实时画面发布为 RTSP 视频流
构建工具	CMake 与 Make	用于在 RK3588 板端编译 C++17 程序
实验记录	CSV、JSONL 和运行日志	用于保存检测结果、ROI 信息、FPS、延迟和阶段耗时

在板端运行时，系统采用 C++17 作为主要实现语言。其原因在于 C++17 便于直接调用 RKNN Runtime、OpenCV 和 GStreamer 等底层库，能够更好地控制线程、队列和内存使用。同时，C++17 程序在嵌入式平台上运行开销较低，更适合实时视频处理场景。

4.2 模型导出与 RKNN 转换实现

本文使用的无人机检测模型首先在 PC/WSL 环境中完成训练，得到 PyTorch 权重文件。为了在 RK3588 上运行，需要将模型从 PyTorch 格式转换为 ONNX 格式，再由 RKNN Toolkit2 转换为 RKNN 模型。整体流程为：

```
best.pt -> best.onnx -> best.rk3588.fp.rknn -> RK3588 NPU 推理
```

在模型转换过程中，本文优先采用 FP 类型 RKNN 模型作为稳定部署版本。针对任务书中的量化优化要求，本文进一步完成了 full INT8 与 hybrid INT8 RKNN 模型转换和板端验证。实验发现，直接 full INT8 虽然可以加载运行，但在公开视频测试中

出现置信度输出塌缩，导致检测框无法稳定产生；保护输出 Sigmoid 至最终输出层的 hybrid INT8 方案能够恢复检测结果。因此，本章将 FPRKNN 作为稳定基线方案，将 hybrid INT8 作为经过板端验证的实验性量化路径。表 4.2 给出了模型迁移过程中涉及的主要文件类型及其作用。

表 4.2 模型文件及作用

文件类型	示例文件	作用说明
PyTorch 权重	best.pt	训练完成后得到的原始模型权重，用于继续训练、验证和导出
ONNX 模型	best.onnx	用于跨框架转换的中间模型格式，便于导入 RKNN Toolkit2
RKNN 模型	best.rk3588.fp.rknn	RK3588 板端实际加载的模型文件，由 RKNN Runtime 调用
检测输出	1×5×8400	单类别无人机检测模型的输出张量，包含边界框参数和置信度

为了适配单类无人机检测任务，后处理模块不再使用 COCO 多类别标签表作为唯一判断依据，而是根据模型输出通道数识别当前模型类别数。当输出张量为 1×5×8400 时，系统将其解析为单类别检测输出，其中 5 个通道分别对应候选框中心点、宽高和置信度信息。检测结果在可视化时统一标注为 **drone**，避免出现类别映射错误。

RKNN 推理模块是系统的核心计算模块，负责完成模型加载、输入预处理、NPU 推理、输出读取和检测结果解析^[7]。本文将该部分封装为独立的检测器类，使固定视频检测程序和实时 RTSP 程序可以复用相同的推理逻辑。

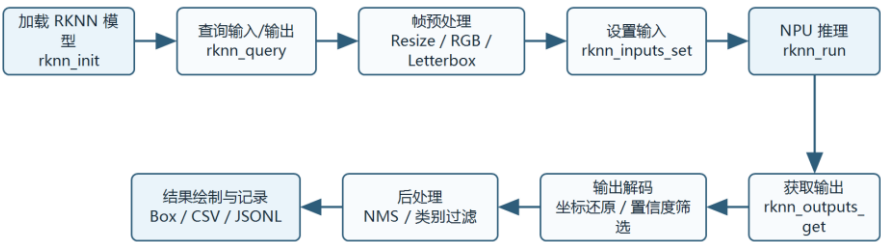


图 4.1 RKNN 推理模块实现流程

RKNN 推理模块的主要步骤如下：

- （1）调用 `rknn_init` 加载 RKNN 模型文件；

- (2) 通过 `rknn_query` 查询模型输入输出张量属性；
- (3) 根据输入张量格式确定模型输入宽度、高度、通道数和数据排布；
- (4) 对原始图像进行缩放、补边和颜色空间转换；
- (5) 调用 `rknn_inputs_set` 设置模型输入；
- (6) 调用 `rknn_run` 在 NPU 上执行模型推理；
- (7) 调用 `rknn_outputs_get` 获取输出张量；
- (8) 对输出张量执行候选框解析、置信度过滤和非极大值抑制；
- (9) 将检测框坐标映射回原始图像尺寸；
- (10) 释放输出内存并返回检测结果。

在实现中，系统会在模型加载阶段打印输入输出张量属性，包括张量维度、数据类型和存储格式。这样做的目的是在模型转换后第一时间发现输入尺寸不匹配、输出通道数异常或数据布局不一致等问题。对于 YOLO 系列模型而言，输出张量格式可能随导出参数而变化，如果不在板端进行显式检查，容易导致后处理阶段解析错误。表 4.3 给出了 RKNN 推理模块中几个关键阶段的功能划分。

表 4.3 RKNN 推理模块关键阶段

阶段	对应功能	实现重点
模型初始化	加载 RKNN 文件并查询张量属性	判断输入尺寸、通道数和输出布局是否符合预期
图像预处理	缩放、补边、颜色转换	保持与训练和导出阶段一致的输入格式
输入设置	将预处理后的图像写入 RKNN 输入	当前稳定路径使用常规输入设置方式
NPU 推理	执行 <code>rknn_run</code>	由 RK3588 NPU 完成主要神经网络计算
输出解析	候选框解码、阈值筛选和 NMS	支持单类无人机检测输出格式
坐标还原	将模型坐标映射回原图	保证绘制框位置与原始画面一致

为了便于后续性能分析，推理模块还提供了可选的阶段耗时统计接口。开启 `profiling` 后，系统可以分别记录图像准备、输入设置、NPU 执行、输出读取、解码和释放等阶段耗时。该接口在正常演示时默认关闭，以减少日志输出和额外开销；在实验分析时再通过环境变量开启。

需要说明的是，本文将 RKNN 推理封装为相对独立的模块。固定视频检测程序

和实时 RTSP 检测程序通过 CMake 复用同一份 yolo_rknn.cpp/yolo_rknn.h 源文件，从而保证模型加载、输入预处理、输出解析和后处理逻辑一致。

4.3 检测程序实现

固定视频检测程序 rk_yolo_video 是本文最先完成板端部署验证的 RK3588 运行路径。该程序的作用是在板端读取本地视频文件，逐帧执行 RKNN 推理，将检测框绘制到原始画面中，并输出带框视频、检测结果 CSV 和 ROIJSONL 文件。在项目仓库中，该程序对应 Thesis Project/rk_yolo_video 目录，核心入口位于 src/main.cpp，RKNN 推理封装主要位于 src/yolo_rknn.cpp。

该程序的命令行接口如下：

```
./rk_yolo_video <input_video> <output_video> [model_path] [score_thresh]
[nms_thresh] [detections_csv] [roi_jsonl]
```

其中，input_video 为输入视频文件，output_video 为带框输出视频，model_path 为 RKNN 模型路径，score_thresh 和 nms_thresh 分别为置信度阈值和非极大值抑制阈值。如果没有显式指定 CSV 和 JSONL 文件路径，程序会根据输出视频文件名自动生成对应的检测记录文件。

固定视频检测程序的输出包括三类数据：

- （1）带框视频，用于直观看到模型是否检测到目标；
- （2）CSV 检测记录，用于统计每一帧的检测结果；
- （3）ROI JSONL 文件，用于兼容后续动态 ROI 或旧系统输出格式。

CSV 文件每一行对应一个检测框，字段包括帧序号、类别编号、类别名称、置信度、左上角坐标、宽度和高度。其字段格式如下：

```
frame_index,class_id,class_name,score,left,top,width,height
```

ROIJSONL 文件则按帧保存检测结果，每一行是一个 JSON 对象，便于后续程序按帧读取目标区域信息。固定视频路径具有较强的可重复性，同一段视频、同一模型和同一阈值可以反复运行得到可比较结果，因此本文将其作为板端固定输入实验的主

要入口。

在固定视频检测路径稳定后，本文进一步实现了实时 RTSP 检测程序 rk_yolo_live_rtsp。该程序既支持 USB 摄像头输入，也支持本地视频文件循环输入。系统读取视频帧后执行目标检测，将检测框叠加到画面上，再通过 GStreamer RTSP Server 发布为 RTSP 视频流，使 PC 端可以使用播放器或 GStreamer 客户端实时查看检测画面。在项目仓库中，该程序对应 Thesis Project/rk_yolo_live_rtsp 目录，核心入口位于 src/main.cpp。

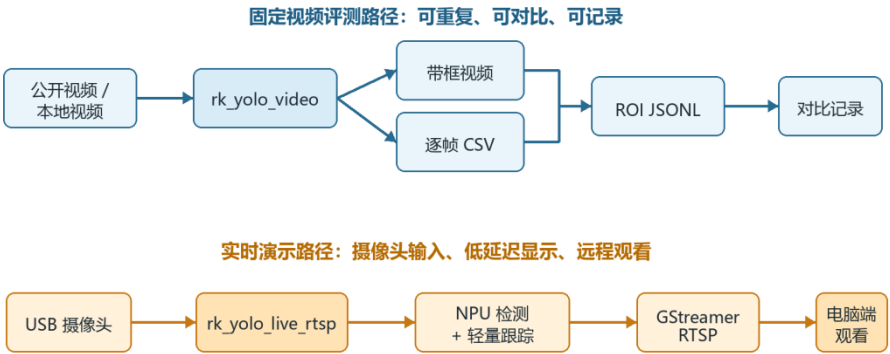


图 4.2 固定视频与实时 RTSP 两条运行路径

实时 RTSP 程序的基本命令行接口如下：

```
./rk_yolo_live_rtsp [device] [model] [mount] [width] [height] [fps] [score] [nms] [port] [detect_every_n]
```

其中，device 可以是 /dev/video* 摄像头设备，也可以是本地视频文件路径；model 为 RKNN 模型路径；mount 为 RTSP 挂载路径；width、height 和 fps 用于指定输出视频参数；score 和 nms 用于设置检测阈值；detect_every_n 用于控制每隔多少帧执行一次完整 NPU 检测。

实时 RTSP 程序内部采用三阶段流水线结构：

- （1）采集线程负责读取摄像头或视频文件，并将最新帧写入有界队列；
- （2）推理线程负责从队列中取出图像，根据调度策略执行 NPU 检测或复用跟踪结果；
- （3）发布线程负责绘制检测框，将画面推送给 RTSP 服务端。

采用有界队列的原因是实时视频系统不能无限积压历史帧。如果推理速度低于输入帧率，系统应优先保证输出画面接近当前时刻，而不是依次处理所有过期帧。因此，队列长度被设计得较小，在必要时丢弃旧帧，以降低端到端延迟。

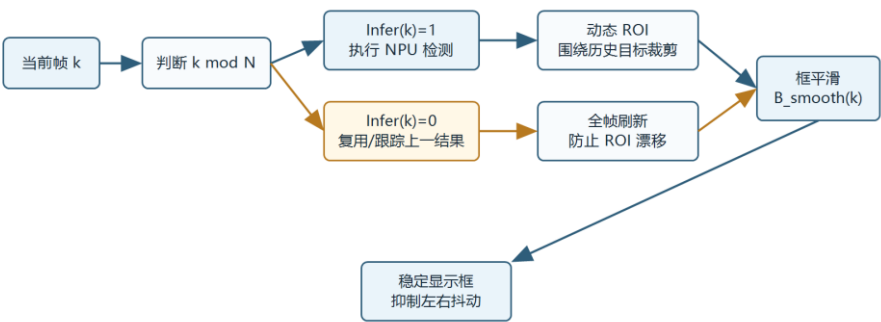
RTSP 输出基于 GStreamer 实现。程序内部创建 `appsrc`，将绘制后的图像帧推入 GStreamer 管线，再通过 H.264 编码器和 RTSP Server 对外发布。PC 端可以通过如下地址查看检测画面：

```
rtsp://<board-ip>:8554/drone
```

对于固定视频循环演示，可以设置 `RK_YOLO_INPUT_LOOP=1`，使程序在视频播放结束后自动回到开头，便于长时间观察和演示。在 USB 摄像头输入场景下，程序还支持对特定 UVC 摄像头执行启动参数调整，例如设置焦距、关闭自动对焦和设置变焦值，以提高画面稳定性。

4.4 检测间隔、动态 ROI 与轻量跟踪实现

在实时检测场景中，如果每一帧都执行完整 NPU 推理，系统可能出现延迟升高或队列积压。为此，本文实现了检测间隔策略，即通过 `detect_every_n` 参数控制完整检测频率。当 `detect_every_n=1` 时，系统每帧执行一次 NPU 检测；当该参数大于 1 时，系统仅在关键帧执行完整检测，其余帧根据轻量跟踪结果更新显示框。



核心思想：减少不必要的 NPU 调用，同时让显示框在跳帧场景下保持连续稳定。

图 4.3 跳帧检测、动态 ROI 与检测框平滑策略

检测间隔策略可表示为：

$$\text{Infer}(k) = \{ 1, \quad k \bmod N = 0; \quad 0, \quad k \bmod N \neq 0 \} \quad (4.1)$$

其中, k 表示当前帧序号, N 表示检测间隔。当 $\text{Infer}(k)=1$ 时, 系统执行完整 NPU 检测; 当 $\text{Infer}(k)=0$ 时, 系统不调用 NPU, 而是使用跟踪或上一帧检测结果进行显示更新。

当跟踪模式为 `TrackMode::kMotion` 时, 程序保存最近两次有效检测结果, 根据检测框中心点位移向量进行线性运动预测, 并对单步位移量进行限幅处理, 防止检测框在目标很小或置信度波动时出现过大跳变。该方式只涉及少量坐标运算, CPU 开销低, 适合作为默认的轻量预测路径。

当跟踪模式为 `TrackMode::kOpticalFlow` 时, 程序在检测框区域内提取稀疏角点, 并通过 Lucas-Kanade 金字塔光流估计目标区域相对位移。实现时采用 OpenCV 中的 `goodFeaturesToTrack` 和 `cv::calcOpticalFlowPyrLK` 函数, 先筛选成功跟踪的特征点, 再计算横向和纵向位移的中值。与直接使用均值相比, 中值位移对少量误匹配点更鲁棒, 因此更适合小目标、低纹理和轻微抖动的视频场景。

轻量跟踪并不替代 NPU 目标检测, 而是在检测间隔期间提供显示框位置估计。系统仍会按照 `detect_every_n` 和周期性全帧刷新策略重新执行完整检测, 以修正跟踪漂移。因此, 该策略的主要作用是降低实时显示中的框跳变和视觉抖动, 而不是将系统扩展为完整的多目标跟踪算法。

为了减少跳帧检测导致的检测框抖动, 系统实现了轻量框平滑策略。其基本思想是将当前检测框与上一帧显示框进行加权融合:

$$B_smooth(k) = \alpha B_detect(k) + (1 - \alpha) B_smooth(k - 1) \quad (4.2)$$

其中, $B_detect(k)$ 表示当前检测框, $B_smooth(k-1)$ 表示上一帧平滑后的显示框, α 为平滑系数。 α 越大, 显示框越接近当前检测结果; α 越小, 显示框越稳定但响应速度越慢。系统通过环境变量提供该参数, 使不同场景下可以根据目标运动速度和画面抖动程度进行调节。

除框平滑外, 系统还实现了动态 ROI 策略。当上一帧已经检测到目标时, 下一次关键帧推理可以优先围绕历史目标区域构造裁剪窗口, 并加入一定边界扩展。这样可以在目标较小的情况下提高局部区域内的目标占比, 同时减少无关背景对检测的干扰。为了避免 ROI 长时间漂移, 系统设置了周期性全帧刷新机制, 使检测过程能够重新

回到完整画面中恢复目标。表 4.4 给出了实时检测策略相关的主要配置项。

表 4.4 实时检测策略配置项

配置项	默认或推荐含义	作用说明
detect_every_n	控制完整检测间隔	在 NPU 调用频率和显示稳定性之间折中
RK_YOLO_TRACK_MODE	轻量运动跟踪模式	在非关键帧估计检测框位置
RK_YOLO_BOX_SMOOTH	是否启用框平滑	减少检测框在相邻帧之间的突变
RK_YOLO_BOX_SMOOTH_ALPHA	平滑系数	控制检测框响应速度与稳定性的平衡
RK_YOLO_DYNAMIC_ROI	是否启用动态 ROI	根据历史检测区域裁剪下一次推理输入
RK_YOLO_ROI_REFRESH	ROI 刷新闻隔	定期恢复全帧检测，降低跟踪漂移风险

4.5 多 context NPU 并行推理实现

为了满足硬件优化方向的研究需求，本文在实时 RTSP 程序中实现了实验性的多 context NPU 并行推理机制。该机制的核心思想是在程序中创建多个 RKNN 检测器实例，每个实例拥有独立的 RKNN context，并由多个推理工作线程并行处理不同帧。这里的多 context 不是多个线程共享同一个 RKNN 上下文，而是每个 worker 独立持有检测器实例，从而减少线程间同步和运行状态竞争。

多 context 模式仅在 detect_every_n=1 时启用。这一限制是有意设计的：当系统已通过 detect_every_n>1 降低 NPU 调用频率时，多 context 并行推理的收益并不明显，反而可能增加调度复杂度。只有在每帧都需要完整检测时，多 context 才能更直接地体现 NPU 并行吞吐能力。

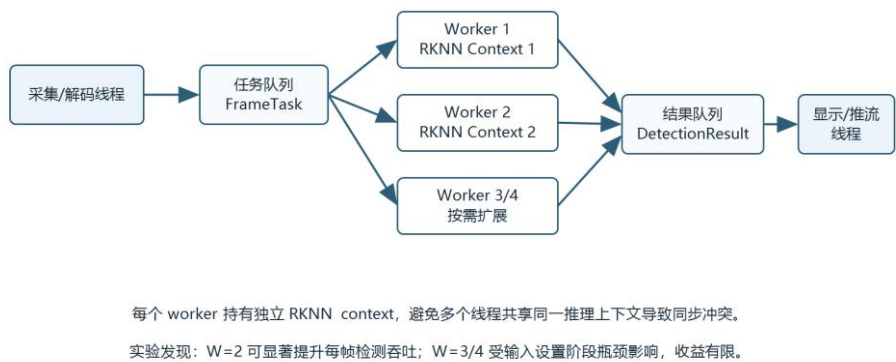


图 4.4 多 context NPU 并行推理实现

多 context 推理的实现流程如下：

- （1）主线程读取 RK_YOLO_INFER_WORKERS 环境变量，确定推理工作线程数量；
- （2）为每个工作线程创建独立的 RKNN 检测器和 RKNN context；
- （3）采集线程将最新帧送入调度队列；
- （4）调度逻辑将待检测帧分配给空闲 worker；
- （5）worker 完成 NPU 推理后将结果写入结果队列；
- （6）发布线程根据帧序号和时间戳选择最新可用结果进行绘制。

表 4.5 单 context 与多 context 实现对比

实现方式	特点	适用场景
单 context	资源占用较低，调度简单，延迟更容易控制	实时演示和稳定运行
双 context	可以重叠多个推理任务，提高每帧检测吞吐	NPU 并行能力验证
三 context 及以上	调度和内存开销进一步增加，收益需要实验确认	扩展性分析，不作为默认配置

多 context 机制使本文不仅能够完成“模型在 RK3588 上部署验证”的基本目标，还能够围绕导师提出的 NPU 多线程方向开展实验。需要注意的是，多 context 的目标并不是简单替代当前实时演示配置，而是提供一条硬件并行优化路线。后续实验需要同时比较吞吐、延迟、CPU 占用和内存占用，才能判断其实际价值。

4.6 日志记录与性能调试接口

为了支撑论文实验分析，系统在实现过程中加入了多层日志记录机制。固定视频检测程序主要输出检测结果 CSV、ROI JSONL 和带框视频；实时 RTSP 程序则输出运行时 FPS、NPU 推理频率、端到端延迟、工作线程耗时和队列状态等信息。

实时程序的常规日志包括：

- （1）`stream_fps`：实际发布的视频帧率；
- （2）`npu_fps`：实际执行 NPU 推理的频率；
- （3）`roi_fps`：动态 ROI 推理频率；
- （4）`work_ms`：单帧主要处理耗时；
- （5）队列丢帧信息：用于判断系统是否出现输入积压；
- （6）当前配置项：包括检测间隔、平滑参数、`worker` 数量和 ROI 开关。

当需要进行更细粒度性能分析时，可以设置：

```
RK_YOLO_PROFILE=1
```

开启该开关后，程序会输出推理阶段的细分耗时，包括预处理、输入设置、NPU 运行、输出读取、检测解码和绘制等。通过这些指标，可以判断系统瓶颈究竟来自 NPU 计算、输入数据传输、后处理还是视频输出。

系统还保留了实验性的 `zero-copy input` 路径，对应环境变量为：

```
RK_YOLO_ZERO_COPY_INPUT=1
```

该路径用于探索 RKNN 输入内存绑定是否能够降低输入设置开销。但根据当前工程状态，`zero-copy` 仍属于实验功能，默认关闭。本文在系统实现章节中仅说明其作为调试接口存在，不将其作为稳定基线方案。这样的写法可以真实反映项目进展，避免把尚未完整验证的优化方向写成成熟结果。

4.7 本章小结

本章完成了从 PC/WSL 端模型导出到 RK3588 板端 C++部署的主要实现工作。实际开发中，最关键的不是单独完成格式转换，而是让 RKNN 输出张量、C++后处理、视频绘制和 RTSP 输出保持一致。

固定视频程序用于确认模型能够稳定产生有效检测框，实时 RTSP 程序用于检查系统在连续输入下的延迟和显示效果；多 context、检测间隔、动态 ROI 和日志接口则为第五章的性能测试提供了可切换的实验条件。

通过本章实现，系统已经从单个模型文件转变为可以在 RK3588 上运行、记录和对比的检测程序。第五章将在统一输入和统一指标下，对这些实现路径进行实验验证。

第五章 系统测试与实验分析

系统能运行并不等于适合实时演示。本章的实验围绕三个更具体的问题展开：模型在公开无人机视频中是否确实能够产生有效检测框；转换后的 RKNN 模型在板端是否稳定；不同检测策略和多 context 配置会怎样影响实际观看流畅度、端到端延迟和资源占用。

由于真实无人机场景受场地和安全条件限制，本文采用公开无人机视频作为固定输入，并结合 RK3588 实时 RTSP 输出进行验证。固定视频保证不同配置之间的可比性，实时 RTSP 则用于观察最终演示时的画面延迟和检测框稳定性。

5.1 实验环境与评价指标

本文实验主要包括 PC/WSL 端模型训练与评估、RK3588 板端固定视频验证以及 RK3588 实时 RTSP 推流验证。PC/WSL 端用于训练 YOLOv10 单类无人机模型，并完成 PyTorch、ONNX 与 RKNN 文件的生成；RK3588 板端用于加载 RKNN 模型并执行视频检测、结果绘制和 RTSP 输出。表 5.1 给出了本章实验涉及的主要环境和输入条件。

表 5.1 实验环境与输入条件

类别	内容	说明
训练主机	Windows 与 WSL 环境	用于 YOLOv10 模型训练、ONNX 导出和 RKNN 转换准备
部署平台	RK3588 开发板	用于加载 RKNN 模型并调用 NPU 推理
部署程序	rk_yolo_video、rk_yolo_live_rtsp	分别用于固定视频检测和实时 RTSP 检测
主要模型	FP RKNN 模型	当前稳定的无人机单类 RKNN 模型
输入视频	公开视频样本	用于短视频验证和较长序列验证
输出内容	带框视频、CSV、ROI JSONL、RTSP 流、运行日志	用于定性观察和定量分析

评价指标分为模型检测指标和系统运行指标两类。模型检测指标主要包括

precision、recall、mAP50 和 mAP50-95，用于反映模型在验证集上的检测效果。系统运行指标主要包括 stream_fps、npv_fps、端到端延迟、CPU 占用、内存占用和丢帧数量，用于评价实时检测系统在 RK3588 上的运行表现。表 5.2 给出了本章使用的主要系统指标含义。

表 5.2 系统运行指标说明

指标	含义	分析作用
stream_fps	实际输出视频流帧率	反映观看端画面流畅程度
npv_fps	实际执行 NPU 推理的频率	反映模型检测刷新强度
端到端延迟	从采集到输出的整体延迟	反映实时响应能力
work_ms	单帧主要处理耗时	用于分析处理链路负载
CPU 占用	程序运行时 CPU 使用比例	反映板端资源压力
RSS 内存	程序常驻内存占用	反映多 context 和缓存开销
dropped_capture / dropped_publish	采集或发布阶段丢帧数量	反映系统是否出现队列积压

需要说明的是，本文当前阶段的重点是嵌入式部署与 NPU 多线程优化，而不是提出新的检测网络结构。因此，本章在讨论模型效果时更强调“模型能否作为系统输入稳定工作”，在讨论系统实验时则更强调“RK3588 上不同运行策略的吞吐与延迟差异”。

为保证实验结果具有可比性，本章实时 RTSP 实验均在同一块 RK3588 开发板和同一套 FP RKNN 模型上完成。每组配置在 RTSP 客户端连接稳定后持续运行不少于 60 s，统计时排除启动初期约 5 s 的初始化波动；stream_fps 和 npv_fps 由程序按固定时间窗口周期性统计，端到端延迟由采集时间戳与发布/绘制时间戳之差得到。固定视频实验则使用相同输入视频和相同阈值重复运行，以便比较不同策略下的输出结果和日志数据。

5.2 模型训练与测试

本文使用 YOLOv10n 作为基础模型，并在单类无人机数据集上进行迁移训练。训练轮数为 50 epoch，输入尺寸为 640，batch size 为 16，训练完成后得到 PyTorch 权

重文件 `best.pt`。该模型随后被导出为 ONNX，并进一步转换为 RK3588 可加载的 RKNN 模型。

训练脚本中保留 `epochs=100` 和 `patience=20` 等配置项，用于支持更长训练和 `early stopping` 策略；本文报告的模型来自实际保存的 50 epoch 训练结果，因此论文中按最终有效训练轮数记录为 50 epoch。表 5.3 给出了训练完成后验证集上的主要指标。

表 5.3 单类无人机模型训练结果

指标	数值
Precision	0.8291
Recall	0.8426
mAP50	0.8901
mAP50-95	0.5095
训练轮数	50
输入尺寸	640 × 640

由表 5.3 可知，当前模型在验证集上具有较高的 mAP50，说明其能够较好地区分无人机目标与背景。mAP50-95 相对低于 mAP50，说明在更严格的定位阈值下，模型的边界框精度仍有提升空间。这与小目标无人机检测任务特点一致：远距离无人机在图像中尺寸较小，轻微的边界框偏移就会对高 IoU 阈值下的评价结果产生较大影响。

在论文表述中，本文不将该训练结果描述为算法创新，而是将其作为后续 RK3588 部署和实时检测实验的模型基础。这样可以避免夸大模型本身贡献，同时突出本课题的系统移植和硬件优化重点。

为了弥补真实无人机飞行场景不易获得的问题，本文引入公开无人机视频作为固定输入测试源。公开视频层的作用不是直接替代完整数据集训练，而是为系统提供可重复的视频输入，使不同程序、不同模型和不同运行策略可以在相同场景下进行比较。

PC/WSL 端首先使用 `best.pt` 对 `anti_uav_fig1.mp4` 进行离线检测。该视频共 130 帧，在置信度阈值 0.35 下，检测结果为：16 帧存在检测框，总检测数为 18，最高置信度为 0.6216。这说明当前模型能够在新的公开无人机视频源中产生目标响应。

随后，本文将模型部署到 RK3588 板端进行固定视频验证。早期直接转换的无人机 RKNN 模型曾在 `rknn_run` 阶段出现运行崩溃。后续通过重新导出 `end2end=False` 的

ONNX 模型，并在 C++ 后处理模块中适配单类输出头 $1 \times 5 \times 8400$ ，恢复了板端运行路径。该过程说明模型部署并不是单纯格式转换，还需要与输出张量结构和后处理逻辑保持一致。表 5.4 给出了 RK3588 固定视频验证结果。

表 5.4 RK3588 固定视频检测结果

视频样本	帧数	有检测帧	检测数	平均推理
短公开视频	130	30	30	109.43 ms
DUT video01	1050	846	846	67.61 ms

由表 5.4 可知，恢复后的无人机 RKNN 模型不仅能够在短视频上运行，也能够较长的 DUT-Anti-UAV 序列上稳定产生检测结果。其中 video01.mp4 共 1050 帧，有 846 帧输出检测框，说明该视频适合作为后续重复实验和论文定性展示的主要公开视频样本。

为更直观展示固定视频和室内演示中的检测效果，本文从公开视频固定输入和室内摄像头演示录屏中选取若干代表帧，如图 5.1 所示。可以看到，系统能够在天空背景、近景无人机以及室内屏幕模拟场景中输出目标类别、置信度和边界框。

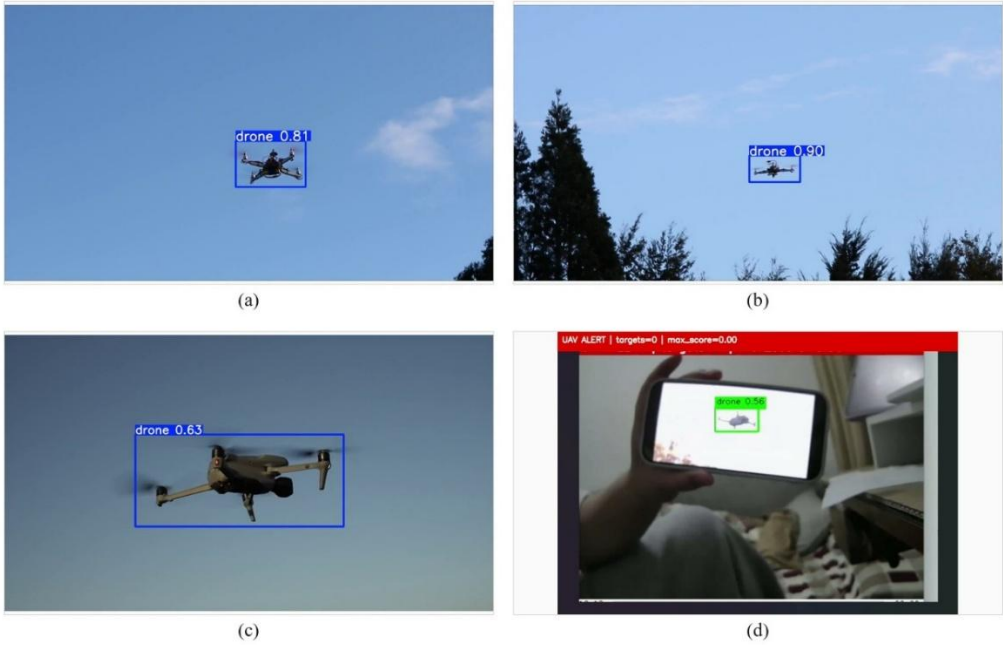


图 5.1 无人机带框检测结果示例

固定视频验证的意义在于，它将模型可用性、RKNN 运行稳定性和输出记录能力统一到同一条可重复路径中。相比只进行实时摄像头演示，固定视频可以保存带框视

频、逐帧 CSV 和 ROI JSONL 文件，更适合论文实验记录和后续误差分析。

在固定视频路径完成验证后，本文进一步测试实时 RTSP 检测程序。首轮实时实验比较了基线全帧检测、多 context 全帧检测和策略优化检测三类配置。其目标不是单纯追求最高帧率，而是观察不同配置在吞吐和延迟之间的差异。表 5.5 给出了首轮实时 RTSP 实验的主要结果。

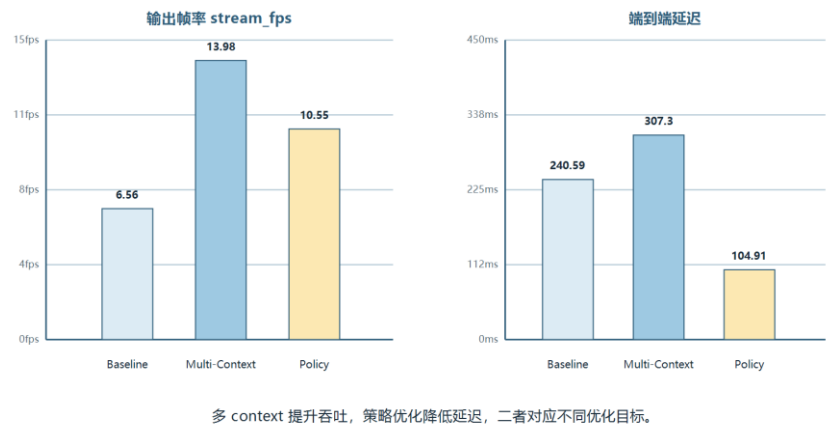


图 5.2 实时 RTSP 首轮配置对比

表 5.5 实时 RTSP 首轮配置实验结果

配置	stream	npu	工作耗时	端到端延迟	采集丢帧	说明
Baseline	6.56	6.56	152.49 ms	240.59 ms	80	单 worker 全帧推理基线
Multi-Context	13.98	13.98	180.16 ms	307.30 ms	0	双 context 全帧推理，吞吐最高
Policy-Optimized	10.55	5.04	N/A	104.91 ms	1	跳帧与复用策略，延迟最低

表中 Policy-Optimized 的平均推理/工作耗时显示为 N/A，是因为采样日志对应的是复用帧，而不是完整 NPU 推理帧。因此该数值不能直接与全帧推理配置比较，但其端到端延迟仍然反映了策略模式下的实时观看响应。

实验结果显示，Multi-Context 配置提高了输出帧率和 NPU 推理频率，但端到端延迟也随之升高。Policy-Optimized 配置没有追求每帧都执行完整检测，而是通过跳帧检测、动态 ROI 和轻量跟踪保持显示连续性，端到端延迟最低。该结果说明实时系统优化不能只看 NPU 吞吐，还需要综合考虑检测刷新频率、队列积压、输出延迟和观看体验。

5.3 检测间隔与框平滑策略实验

在实时观看过程中，检测框曾出现左右抖动现象。分析认为，这与检测间隔较大、轻量跟踪漂移以及框平滑未启用有关。为此，本文比较了 detect_every_n 和 box smoothing 对实时显示稳定性的影响。

首先，在无目标或弱目标实时场景下比较了 detect_every_n=2、3、4 三种策略。实验结果如表 5.6 所示。

表 5.6 检测间隔策略实验结果

配置	stream	npu	延迟	CPU	RSS	说明
N=2	10.02	6.68	175.20 ms	43.0%	132.37 MB	NPU 刷新强度较高，但延迟较高
N=3	10.91	4.96	107.18 ms	37.1%	127.36 MB	帧率、延迟和资源占用较均衡
N=4	9.54	3.82	95.54 ms	33.5%	127.97 MB	延迟较低，但刷新强度下降

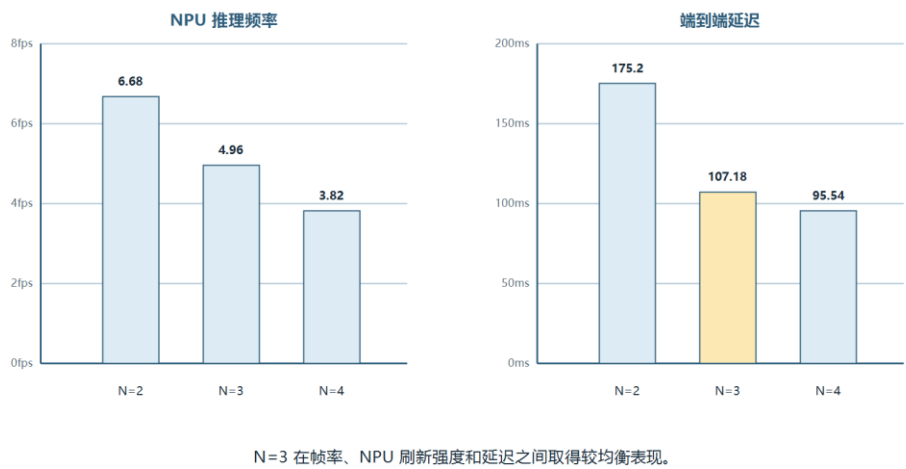


图 5.3 检测间隔策略性能变化

由表 5.6 可知，随着 detect_every_n 增大，NPU 推理频率降低，端到端延迟总体下降，但目标刷新能力也随之降低。N=4 虽然延迟较低，但输出帧率和检测刷新强度均下降；N=2 虽然检测更频繁，但延迟较高。因此，在该轮实验中 N=3 可作为无目标实时场景下的较优折中点。

后续在公开视频 video01.mp4 的目标场景中再次比较 N=3 和 N=2，并加入框平滑。结果表明，N=2 配合 box_smooth=on 后可以减轻检测框左右抖动，虽然 NPU 调用频率提高，但输出帧率仍保持在约 10 FPS。表 5.7 给出了该轮消融实验结果。

表 5.7 跳帧检测与框平滑消融实验结果

配置	stream	npu	延迟	工作耗时	观察结果
N=3，平滑关闭	9.95	3.32	102.60 ms	101.99 ms	框抖动较明显
N=3，平滑开启	9.93	3.31	108.05 ms	107.03 ms	抖动减轻，但刷新频率不变
N=2，平滑开启	10.36	5.18	111.41 ms	110.76 ms	检测框更稳定，实时性仍可接受

综合两轮实验可知，若系统处于纯实时观看、目标不明显或演示场景较简单时，detect_every_n=3 可以作为低延迟默认配置；若视频中存在无人机目标且需要更稳定的框位置，detect_every_n=2 加框平滑更适合作为当前演示配置。

5.4 性能优化与瓶颈分析

根据课题任务要求，RK3588 NPU 多 context 并行是本文重点验证的硬件优化方向之一。为验证这一方向，本文在实时 RTSP 程序中加入多 context 推理机制，即创建多个 RKNN 检测器实例，每个 worker 拥有独立 RKNN context，并由调度队列分配待检测帧。表 5.8 给出了多 context NPU 并行实验结果。

表 5.8 多 context NPU 并行推理实验结果

配置	N	W	stream	npu	工作耗时	端到端延迟	丢帧	说明
A	1	1	6.80	6.80	141.30 ms	254.88 ms	364 capture / 0 publish	单 context 全帧推理基线
B	1	2	13.71	13.71	124.91 ms	215.25 ms	0 capture / 0 publish	双 context 全帧推理，吞吐提升明显
C	2	1	12.16	6.75	128.32 ms	188.97 ms	69 capture / 0 publish	检测更频繁，延迟适中
D	3	1	13.40	4.45	87.81 ms	92.16 ms	0 capture / 0 publish	低延迟实时观看配置

实验结果表明，单 context 每帧检测不适合作为实时默认配置。其 stream_fps 仅为 6.80 FPS，端到端延迟为 254.88 ms，并出现 364 帧采集丢帧。双 context 每帧检测将 stream_fps 和 npu_fps 提升到 13.71 FPS，同时消除了采集和发布丢帧，说明多 context 机制能够显著提升每帧检测场景下的吞吐能力。

从同一公开视频输入下的策略对比看，`detect_every_n=2` 在保持较高检测刷新频率的同时，将端到端延迟降低到 188.97 ms；`detect_every_n=3` 进一步将平均端到端延迟降低到 92.16 ms，并保持 13.40 FPS 输出。该结果说明，实时观看场景不能只追求每帧检测，还需要结合检测间隔、动态 ROI、轻量跟踪和框平滑来控制端到端延迟。

因此，本轮矩阵实验形成两条结论。第一，从硬件优化角度看，双 context 是当前配置下最有效的 NPU 并行实验方案，可以作为导师要求的 NPU 多线程方向的主要数据支撑。第二，从最终演示角度看，`detect_every_n=3` 配合动态 ROI、运动预测和框平滑更适合作为低延迟实时观看配置；若更强调目标检测刷新频率，则可切换到 `detect_every_n=2`。

前一轮实验说明，多 context 并行可以提升每帧检测吞吐，但三 context 继续增加 worker 后收益并不明显。为了判断限制来自 NPU 计算本身，还是来自输入设置、输出获取或后处理环节，本文加入 `RK_YOLO_PROFILE` 开关，对 `prepare`、`rknn_inputs_set`、`rknn_run`、`outputs_get`、`decode` 和 `render` 等阶段分别计时。表 5.9 给出了单 context 和双 context 条件下的阶段耗时对比。

表 5.9 RKNN 推理阶段耗时分析

阶段	N=1, W=1	N=1, W=2
Stream FPS	8.57	10.53
NPU FPS	8.57	10.53
端到端延迟	255.52 ms	179.52 ms
总推理工作耗时	114.48 ms	113.96 ms
Prepare	3.62 ms	1.42 ms
rknn_inputs_set	62.01 ms	63.78 ms
rknn_run	45.82 ms	45.72 ms
rknn_outputs_get	0.49 ms	0.54 ms
Decode + NMS	2.40 ms	2.48 ms
Render	0.15 ms	0.13 ms

由表 5.9 可知，双 context 并没有降低单次推理本身的总工作耗时，两个配置下总推理耗时均约为 114 ms；其吞吐提升主要来自不同推理任务之间的重叠执行，而不是单帧 NPU 计算速度变快。

从阶段分解看，rknn_inputs_set 耗时约 62 至 64 ms，高于 rknn_run 的约 46 ms；Decode + NMS 和 Render 仅为毫秒级或亚毫秒级，说明后续优化应优先关注输入数据传输、预处理和内存布局，而不是继续压缩画框绘制开销。

基于阶段耗时分析，本文进一步比较 OpenCV 默认预处理、RGA 颜色转换与缩放路径，以及 RKNN zero-copy 输入路径。三种路径均通过环境变量控制，默认关闭实验路径，以避免影响稳定演示。

其中，RGA 路径通过 RK_YOLO_PREPROCESS=rga_cvt_resize 启用；zero-copy 路径通过 RK_YOLO_ZERO_COPY_INPUT=1 启用，使用 rknn_create_mem 创建 RKNN 管理的输入内存，并通过 rknn_set_io_mem 绑定到模型输入。固定视频硬件输入路径实验采用同一公开视频片段进行对比。

固定视频硬件输入路径实验显示，OpenCV baseline 的 total_work_ms 为 146.60 ms，RGA cvt+resize 路径为 147.17 ms，二者检测 CSV 完全一致，但 RGA 路径在本轮实验中未形成明显加速；zero-copy input 路径将输入更新阶段降至 0.45 ms，total_work_ms 降至 137.07 ms，检测 CSV 也与 OpenCV 路径一致，但 rknn_run 阶段上升至 92.62 ms，说明部分输入转换或同步开销可能转移到了 NPU 执行阶段。因此，本文将 RGA 和 zero-copy 作为已经完成可切换验证的硬件优化探索，而不是默认稳定路径。

Zero-copy 路径在本轮固定视频实验中将输入更新阶段降低到 0.45 ms，检测 CSV 也与 OpenCV 路径完全一致；但 rknn_run 阶段上升到 92.62 ms，说明部分输入转换或同步开销可能转移到了 NPU 执行阶段。尽管本轮 total_work_ms 降低到 137.07 ms，但其机制仍需结合实时 RTSP 链路继续验证。

因此，本文将 RGA 和 zero-copy 作为已经完成可切换验证的硬件优化探索，而不是默认稳定路径。当前演示仍建议保持 OpenCV 预处理和常规输入设置；RGA、zero-copy 和 hybrid INT8 量化路径则用于说明系统已经围绕输入传输、硬件预处理和模型量化进行了分层实验。

为进一步对应任务书中“视频采集、RGA 硬件预处理、NPU 推理、后处理的异步并行流水线”要求，本文在 zero-copy 输入路径之外补充了 taskbook full-RGA strict 验证配置。该配置设置 RK_YOLO_PIPELINE=1、RK_YOLO_PIPELINE_STAGED=1、RK_YOLO_PREPROCESS=rga_cvt_resize 、 RK_YOLO_RGA_LETTERBOX=1 和

RK_YOLO_REQUIRE_RGA=1, 其中 RK_YOLO_REQUIRE_RGA 用于禁止 RGA 失败后回退 OpenCV。板端日志显示 preprocess=rga_cvt_resize、rga_letterbox=on、rga_required=on, 并输出 rga_api version 1.10.1_[10]。

在 anti_uav_fig1.mp4 的 130 帧验证中, 该配置保持 30 帧出现检测结果和 14 次报警事件, 说明 RGA 已参与颜色转换、缩放和 letterbox 输入构造, 能够作为 RGA 硬件预处理流水线的工程验证结果。该配置的 mean total work time 约为 190.133 ms, 端到端速度不优于稳定基线方案, 因此本文将其作为任务书合规性和硬件预处理功能验证, 而不是最终性能推荐配置。

RGA 硬件预处理的加速效果需要在完整硬件流水线中才能充分体现。RGA 的优势在于与 MPP 硬件解码器配合, 使用物理连续内存减少 CPU 拷贝。当前实验使用 OpenCV 读取已解码视频帧, 数据位于 CPU 虚拟地址空间, RGA 仍需进行缓冲区访问和数据搬运, 相关开销会抵消部分硬件计算节省的时间。只有在“解码→RGA 预处理→NPU 推理”全链路使用物理连续内存的流水线中, RGA 才更可能发挥真正的零拷贝优势。本文的 RGA letterbox 验证实验已确认检测结果与 OpenCV 基线一致, 说明 RGA 功能链路正确; 完整零拷贝链路集成仍作为后续优化方向。

将本章结果放在一起看, 可以发现当前系统的最优配置并不是单纯由最高 NPU FPS 决定的。训练和公开视频验证首先说明, 单类无人机模型可以作为 RK3588 检测系统的有效输入; 板端固定视频实验进一步确认, 恢复后的 end2end=False RKNN 模型能够稳定运行。

固定视频和实时 RTSP 两条路径承担的任务不同。固定视频输入更适合复现实验、生成带框视频和分析 CSV 结果; RTSP 实时路径则更接近最终演示, 端到端延迟、画面流畅性和检测框稳定性比单次推理耗时更直接影响观看效果。

多 context 实验显示, 双 context 在每帧检测时能够把 stream_fps 和 npu_fps 从 6.80 提升到 13.71, 并消除采集丢帧, 这是本文最明确的 NPU 并行收益。但实时演示并不一定要每帧都调用 NPU。结合检测间隔、动态 ROI、运动预测和框平滑后, detect_every_n=3 在本轮实验中取得约 13.40 FPS 和 92.16 ms 平均端到端延迟, 因此更适合作为默认观看配置。

RGA、zero-copy 和 INT8 实验则提醒我们, 硬件加速路径不能只看能否完成端到端验证, 还要看端到端结果。RGA 路径已经验证了视频采集、RGA 预处理、NPU 推

理和后处理链路，但当前虚拟地址输入和编码输出仍会带来额外开销；hybrid INT8 恢复了检测结果，却没有稳定超过 FP 稳定方案。

因此，本文推荐的实时演示配置以 FPRKNN 模型、detect_every_n=3、动态 ROI、运动预测和框平滑为主；双 context 更适合作为展示 NPU 并行吞吐能力的实验配置，RGA 和 hybrid INT8 则作为已经验证但仍需继续优化的硬件路径保留。表 5.10 给出了当前阶段推荐配置。

表 5.10 当前推荐配置

使用目标	推荐配置	理由
实时观看演示	detect_every_n=3 , infer_workers=1 , box_smooth=on , dynamic_roi=on , track_mode=motion	平均 stream_fps 约 13.40，端到端延迟约 92.16 ms，且无采集和发布丢帧
目标刷新优先	detect_every_n=2 , infer_workers=1 , box_smooth=on , dynamic_roi=on , track_mode=motion	NPU 刷新频率约 6.75 FPS，适合目标运动较快或更重视检测新鲜度的场景
NPU 多线程实验	detect_every_n=1, infer_workers=2	相比单 context 全帧检测,stream_fps 由 6.80 提升到 13.71，可体现双 context 并行收益
硬件优化探索	zero-copy 输入、RGA cvt+resize、hybrid INT8 量化	均已完成可切换验证；hybrid INT8 可恢复检测结果，但默认演示仍保持稳定 OpenCV 与 FP RKNN 路径

表 5.10 中的推荐配置基于 RK3588 开发板、公开无人机视频输入、FP RKNN 单类无人机模型以及本文当前后处理实现得到。若后续更换为 USB 摄像头实时输入、INT8 模型、RGA 完整硬件预处理路径或不同分辨率输入，推荐参数仍需重新测试。

需要特别说明的是，full INT8 量化和完整 RGA 硬件预处理目前不应写成已经替代默认运行方案的稳定优化结果。根据当前实验，FP RKNN 模型和 OpenCV 预处理仍是稳定基线方案；RGA cvt+resize、zero-copy 输入和 hybrid INT8 路径已经完成可切换验证，其中 hybrid INT8 能够在保护少量输出层的条件下恢复检测结果，但仍应表述为硬件优化探索，而不是默认演示配置。

从硬件优化路径看，本文已完成多 context NPU、RGA 预处理、RGA 帧缩放、RGA 发布格式、RGA letterbox、RKNN zero-copy 输入和软件告警等多种可切换实验。不同优化项的收益并不相同，因此本文不简单以是否使用硬件加速作为默认配置依据，

而是结合端到端延迟、输出帧率、检测一致性和演示稳定性进行选择。

综合来看，本文最终推荐的实时演示方案并不是单一硬件模块的极限配置，而是检测间隔、轻量跟踪、框平滑和动态 ROI 的组合策略。多 context NPU 和 RGA 相关路径则用于支撑硬件优化实验，说明系统已经围绕 RK3588 的 NPU 和 RGA 资源进行了分层验证。Zero-copy 和 NV12 发布路径虽然已经实现并验证可运行，但由于端到端收益不稳定，本文将其作为后续优化方向而非默认配置。

5.5 长时间运行稳定性测试

为回应任务书中“重点测试长时间运行稳定性”的要求，本文在前述短时配置对比之外补充了两组稳定性测试。第一组为独立 NPU 推理稳定性测试，用于观察模型加载、输入设置、NPU 执行和输出读取在长时间循环运行中的可靠性；第二组为 RTSP 实时流测试，用于验证多线程实时链路在持续推流过程中的内存占用、丢帧和报警事件记录情况。两组测试分别从推理核心和实时链路两个角度进行互补验证。

表 5.11 长时间运行稳定性测试结果

测试项目	运行规模	关键结果与结论
1 小时独立推理	3605.6 s; 24,700 次 NPU 推理	平均推理耗时 52.38 ms; RSS 约 105.6~107.2 MB, 最终 104.2 MB; 0 错误、0 崩溃, 无明显内存泄漏。
10 分钟 RTSP 实时流 综合判断	597 s; 约 4,100 次 NPU 推理 累计约 28,800 次 NPU 推理	发布丢帧 0; RSS 约 125~127 MB; 记录 341 次报警事件; 实时链路运行稳定。 未观察到持续性能退化或内存增长趋势, 能够满足长时间运行稳定性验证要求。

由表 5.11 可知，独立推理测试在约 1 小时内完成 24,700 次 NPU 推理，未出现错误和崩溃，RSS 内存占用保持稳定；RTSP 实时流测试持续约 10 分钟，未出现程序崩溃和发布丢帧。两组结果表明，系统在长时间连续推理和实时推流场景下均表现稳定，能够支撑任务书中关于长时间运行稳定性的验证要求。

5.6 本章小结

本章用固定视频和实时 RTSP 两条路径检验了系统是否真正可用。训练指标说明模型具有基本检测能力，公开视频固定输入验证了模型在新场景中能够产生有效检测

框，板端实验则进一步确认恢复后的 RKNN 模型可以在 RK3588 上稳定运行。

实时 RTSP 测试表明，双 context 适合展示每帧检测下的 NPU 吞吐能力，而检测间隔、动态 ROI、运动预测和框平滑更适合改善实时观看体验。阶段耗时分析把问题进一步拆开：当前瓶颈并不只在 rknn_run，也与 RKNN 输入设置、NPU 执行、后处理和实时调度的组合开销有关。

在 INT8 量化方面，直接 full INT8 导致公开视频中无法稳定产生有效检测框，hybrid INT8 恢复了检测能力但尚未形成默认加速路径。RGA、zero-copy 和 hybrid INT8 实验均已完成阶段性验证，但它们对端到端延迟和检测稳定性的影响并不一致，因此本文没有把这些路径简单写成默认优化结果。

总体来看，第五章验证了当前系统的稳定主线，也记录了硬件优化过程中尚未完全收敛的部分。第六章将在这些实验结论基础上，对全文工作和后续限制进行总结。

总结与展望

本文完成了单类无人机检测模型在 RK3588 平台上的迁移、部署和实验验证。围绕这一目标，本文先完成模型导出、ONNX/RKNN 转换、板端加载和输出张量解析，再在 C++17 程序中实现固定视频检测与实时 RTSP 检测两条运行路径，使检测结果能够以带框视频、日志和实时视频流形式输出。

在系统优化方面，本文重点比较了检测间隔、动态 ROI、轻量跟踪、框平滑、多 context 并行、RGA 预处理、zero-copy 输入和 INT8 量化等策略。实验表明，双 context 能够明显提升每帧检测时的 NPU 吞吐；而在实时观看场景下，detect_every_n=3 配合动态 ROI 和框平滑更能兼顾刷新率、延迟和画面稳定性。RGA 和 hybrid INT8 路径已经完成验证，但尚未稳定超过 FP RKNN 稳定方案，因此本文将 FP 模型作为最终演示的默认配置。

然而，本文仍存在不足之处。回到整个项目本身，最大的不足仍然是真实无人机飞行测试不够。公开视频和室内演示能证明程序链路可以运行，但它们不能完全替代户外场景。真实飞行时，无人机距离、背景、光照和姿态变化都更复杂，后续如果条件允许，应在安全合规的场地补充不同距离和不同背景下的测试。

技术上，INT8 量化是本项目遇到调试问题最明显的一条线。full INT8 模型可以加载，也能完成推理调用，但在 130 帧公开视频测试中置信度整体塌缩，几乎无法稳定产生有效检测框。后来采用保护输出 Sigmoid 至最终输出层的 hybrid INT8 方案后，检测结果恢复了，但速度并没有稳定优于 FP 稳定方案。这个结果说明，小目标无人机检测对量化误差非常敏感，后续需要结合校准数据、输出分布和后处理阈值继续分析。

参考文献

- [1] REDMON J, FARHADI A. YOLOv3: An Incremental Improvement[EB/OL]. (2018)[2026-04-27]. <https://arxiv.org/abs/1804.02767>.
- [2] BOCHKOVSKIY A, WANG C Y, LIAO H Y M. YOLOv4: Optimal Speed and Accuracy of Object Detection[EB/OL]. (2020)[2026-04-27]. <https://arxiv.org/abs/2004.10934>.
- [3] WANG A, CHEN H, LIU L, et al. YOLOv10: Real-Time End-to-End Object Detection[EB/OL]. (2024)[2026-04-27]. <https://arxiv.org/abs/2405.14458>.
- [4] ZHU X, LYU S, WANG X, et al. TPH-YOLOv5: Improved YOLOv5 Based on Transformer Prediction Head for Object Detection on Drone-captured Scenarios[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops. 2021: 2778-2788.
- [5] HUANG B, LI J, CHEN J, et al. Anti-UAV410: A Thermal Infrared Benchmark and Customized Scheme for Tracking Drones in the Wild[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2024, 46(5): 2852-2865.
- [6] TIJTGAT N, VAN RANST W, GOEDEME T, et al. Embedded Real-Time Object Detection for a UAV Warning System[C]//Proceedings of the IEEE International Conference on Computer Vision Workshops. 2017: 2110-2118.
- [7] ROCKCHIP. RKNN Toolkit2 and RKNPU2 Documentation[EB/OL]. [2026-04-27]. <https://github.com/airockchip/rknn-toolkit2>.
- [8] ULTRALYTICS. Ultralytics YOLO Repository[EB/OL]. [2026-04-27]. <https://github.com/ultralytics/ultralytics>.
- [9] OPEN NEURAL NETWORK EXCHANGE. ONNX Documentation[EB/OL]. [2026-04-27]. <https://onnx.ai/onnx/>.
- [10] OPENCV TEAM. OpenCV Documentation[EB/OL]. [2026-04-27]. <https://docs.opencv.org/>.
- [11] GSTREAMER PROJECT. GStreamer Documentation[EB/OL]. [2026-04-27]. <https://gstreamer.freedesktop.org/documentation/>.
- [12] ROCKCHIP. RK3588 Product Specification[EB/OL]. (2022-09-26)[2026-04-28]. https://www.rock-chips.com/a/en/products/RK35_Series/2022/0926/1660.html.

- [13] NVIDIA. Jetson Nano Technical Specifications[EB/OL]. [2026-04-28].
- [14] NVIDIA. Jetson[EB/OL]. [2026-04-28]. <https://developer.nvidia.com/jetson>.
- [15] LUCAS B D, KANADE T. An Iterative Image Registration Technique with an Application to Stereo Vision[C]//Proceedings of the 7th International Joint Conference on Artificial Intelligence. 1981, 2: 674-679.
- [16] SHI J, TOMASI C. Good Features to Track[C]//Proceedings of IEEE Conference on Computer Vision and Pattern Recognition. 1994: 593-600.
- [17] JACOB B, KLIGYS S, CHEN B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 2704-2713.
- [18] SANDLER M, HOWARD A, ZHU M, et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 4510-4520.
- [19] WOJKE N, BEWLEY A, PAULUS D. Simple Online and Realtime Tracking with a Deep Association Metric[C]//2017 IEEE International Conference on Image Processing. 2017: 3645-3649.
- [20] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. Programming Languages -- C++: ISO/IEC 14882:2017[S]. Geneva: ISO, 2017.
- [21] LI J, YE J. Edge-YOLO: Lightweight Infrared Object Detection Method Deployed on Edge Devices[J]. Applied Sciences, 2023, 13(7): 4402.
- [22] 何宇豪, 易明发, 周先存, 等. 基于改进的 Yolov5 的无人机图像小目标检测[J]. 智能系统学报, 2024, 19(3): 635-645.
- [23] 韩俊, 袁小平, 王准, 等. 基于 YOLOv5s 的无人机密集小目标检测算法[J]. 浙江大学学报(工学版), 2023, 57(6): 1224-1233.
- [24] 何植仟, 曹立杰. UAVAI-YOLO: 无人机航拍图像的小目标检测模型[J]. 智能科学与技术学报, 2024, 6(2): 262-271.
- [25] 廖嘉仪. 一种面向嵌入式边缘计算设备的反无人机目标检测方法[J]. 智能制造, 2025(2): 43-49.

致谢

本课题的完成离不开指导教师在研究方向、系统设计和论文撰写方面给予的指导与帮助。在项目推进过程中，导师对 RK3588 嵌入式平台、NPU 多线程优化和系统级实验提出了明确要求，使本文能够从单纯模型部署进一步扩展到工程化实时系统设计与性能分析。

同时，感谢同学和实验环境提供者在模型、数据、开发板调试和测试场景方面给予的支持。本文所涉及的模型转换、板端部署、实时视频推流和公开数据验证经历了多轮调试，相关讨论和协助对系统最终完成稳定运行具有重要帮助。最后，感谢家人和朋友在毕业设计期间给予的理解与支持。

作者：朱奕澄

完成地点：上海大学

2026 年 5 月 22 日

